Playing Balatro with LLM-Guided Deep Reinforcement Learning

Anonymous

***Abstract*—Roguelike deckbuilding games pose unique challenges for reinforcement learning: combinatorial action spaces, long planning horizons, and sparse rewards make pure RL difficult to converge, while the strategic knowledge needed for effective play is well-documented in human-authored game guides. We present an LLM-guided RL framework for a simplified version of Balatro, a roguelike deckbuilding poker game, that leverages existing game guides through a retrieval-augmented LLM to assist RL agents in knowledge-intensive decisions where exploration alone is insufficient. Our approach demonstrates the potential of combining LLMs with RL to directly exploit human game knowledge, offering a practical path for training game AI in domains characterized by combinatorial complexity and sparse feedback.**

***Index Terms*—reinforcement learning, LLM action prior, hesitation gate, card games, roguelike deckbuilder, Balatro, action masking**

I. Introduction

Reinforcement learning has achieved superhuman performance in games ranging from Atari [1] to StarCraft II [2], yet roguelike deckbuilding games remain largely unexplored. These games combine stochastic card draws, combinatorial action spaces, and—crucially—*knowledge-intensive decisions* that depend on understanding complex interactions among hundreds of game elements. While prior card game research has focused on competitive settings such as poker [3] and Hanabi [4], single-player deckbuilders pose a distinct challenge: the strategic knowledge needed (e.g., which modifier cards synergize) is abundantly documented in community guides but prohibitively expensive to discover through exploration alone.

We study **Balatro**, a roguelike deckbuilding poker game whose different phases demand different reasoning capabilities (Fig. 1). The hand-play phase—selecting which cards to play from an 8-card hand—is a combinatorial optimization problem amenable to standard RL. The joker selection phase—choosing which modifier cards to equip—requires strategic game knowledge about effect synergies that pure RL struggles to acquire.

This asymmetry motivates our core idea: *use LLMs to inject human game knowledge exactly where RL falls short*. We treat a retrieval-augmented LLM [5] as a Bayesian action prior [6] and incorporate it into a PPO agent via variational inference. To avoid unnecessary LLM overhead, we introduce a *hesitation gate* that activates the prior only when the policy's softmax output exhibits low confidence—effectively implementing the principle "ask for help when uncertain."

Our contributions are:

- A **multi-phase RL framework** for Balatro with phase-specific agents for hand-play and joker selection.

Fig. 1. The hand-play phase in Balatro. The player is dealt 8 cards and must select cards to play as a poker hand or discard. The left panel shows the current blind target, equipped Joker, detected hand type with base scoring values, remaining hands/discards, and current funds.

- A comparison of **DDQN and PPO** with bitmask action encoding and action masking on the combinatorial hand-play phase.
- A **hesitation-gated LLM prior** that selectively incorporates game strategy knowledge via RAG, reducing LLM queries as the policy gains confidence.
- A pure-Python game engine recreating core Balatro mechanics for fast, large-scale RL training.

II. Balatro: Game Overview

In Balatro, the player begins each run with a standard 52-card deck and scores chips by playing poker hands to defeat escalating challenges. A run progresses through 8 *Antes*, each containing 3 *Blinds* (Small, Big, and Boss) with progressively higher chip thresholds.

A. Core Game Phases

Hand Play. The player is dealt 8 cards from their deck with limited *hands* (typically 4) and *discards* (3) per blind. Each turn, the player selects 1–5 cards to either play as a poker hand (consuming a hand) or discard and redraw replacements (consuming a discard). The strategic tension lies in deciding whether to play an available hand immediately or invest discards to improve hand quality.

Scoring. The score for a played hand is computed as:

$$\text{Score} = \underbrace{(B_c + C_c + \Delta_c)}_{\text{Chips}} \times \underbrace{(B_m + \Delta_m) \times \prod_j X_m^{(j)}}_{\text{Multiplier}} \tag{1}$$

where B_c, B_m are base chips and multiplier from the poker hand type (Table I), C_c is the sum of chip values of scoring

TABLE I
POKER HAND TYPES WITH BASE SCORING VALUES (LEVEL 1). PLANET CARDS CAN LEVEL UP EACH HAND TYPE, PERMANENTLY INCREASING ITS BASE VALUES.

Hand Type	Base Chips	Base Mult
High Card	5	1
Pair	10	2
Two Pair	20	2
Three of a Kind	30	3
Straight	30	4
Flush	35	4
Full House	40	4
Four of a Kind	60	7
Straight Flush	100	8
Five of a Kind	120	12

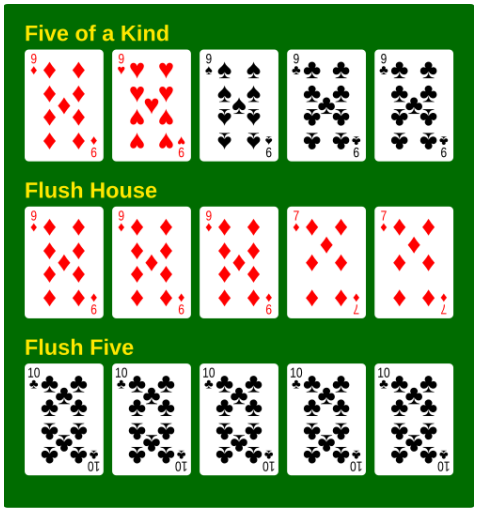

Fig. 2. Balatro introduces special poker hands beyond the standard ranking. *Five of a Kind*: five cards of the same rank. *Flush House*: a Full House where all cards share a suit. *Flush Five*: five identical-rank cards of one suit. These hands yield significantly higher base scores.

cards, and $\Delta_c, \Delta_m, X_m^{(j)}$ are modifiers applied by the j-th Joker. Critically, Jokers are evaluated *left to right*, with multiplicative Jokers (X_m) applied immediately to the running multiplier—making Joker *ordering* a non-trivial strategic consideration.

Shop Phase. After defeating each blind, the player enters a shop where they can spend earned dollars to purchase Jokers, consumables including Tarot cards (22 types), Planet cards (12 types), and Spectral cards (18 types), as well as booster packs and Vouchers (32 types).

Blind Selection. Before each blind, the player may choose to skip the Small or Big blind for a tag bonus, or face a Boss Blind with a special debuff effect (e.g., all Hearts debuffed, face-down cards, reduced hand size). The game features 30 distinct blind types.

B. Jokers: The Key Strategic Element

The game features 150 unique Joker cards—persistent modifiers that alter scoring in qualitatively different ways. The player may hold up to 5 Jokers at a time, and can buy, sell, or replace them at the shop between rounds. Critically, Jokers are evaluated *left to right* during scoring (Eq. 1), so their *positional order* directly affects the result: placing a multiplicative Joker after an additive one yields a different score than the reverse. This means the joker management problem involves not only *which* Jokers to hold but also *in what order* to arrange them—creating a combinatorial explosion. With 150 Joker types, choosing up to 5 from the available pool yields $\sum_{k=0}^{5} \binom{150}{k} > 5.9 \times 10^8$ subsets, each with up to $5! = 120$ orderings.

Our engine implements 30 representative Jokers spanning several archetypes: *flat bonus* (e.g., $+4$ Mult), *conditional* (e.g., $+8$ Mult if the hand contains a Pair), *suit-based* (e.g., $+3$ Mult per Diamond scored), *multiplicative* (e.g., $\times 3$ Mult if all held cards are Spades/Clubs), and *scaling* (e.g., $+15$ Chips per Straight played, cumulative across rounds). Identifying synergies among Jokers and their optimal ordering—the central strategic challenge—requires knowledge that is well-documented in community guides but expensive to discover through trial and error.

C. Challenges for RL

Balatro presents five distinguishing challenges that motivate our design choices: (1) **Combinatorial action space**: selecting k from 8 cards yields $\sum_{k=1}^{5} \binom{8}{k} = 218$ subsets, doubled for play/discard—addressed by bitmask encoding with action masking; (2) **Nonlinear reward**: the multiplicative scoring formula means small changes in card selection can produce order-of-magnitude score differences; (3) **Multi-phase planning**: hand-play and joker selection require different agent architectures; (4) **Long horizon**: 9 rounds with escalating chip requirements and sparse terminal reward; (5) **Knowledge-intensive decisions**: optimal joker selection *and ordering* depends on complex interactions among 150 joker types—motivating our LLM-guided approach.

D. Our Simplified Game Model

The full Balatro game includes shop management, blind selection, consumable usage, and multi-ante progression. To focus on the core RL challenges, we construct a simplified two-phase game model:

- **Joker Selection Phase**: At the start of each run, 5 Jokers are randomly sampled from the full pool of 30 implemented types and offered to the agent, who selects which ones to equip (up to 5 slots).
- **Hand-Play Phase**: The agent plays 9 consecutive rounds, each consisting of a deal of 8 cards followed by a sequence of play/discard decisions with limited hands and discards. The equipped Jokers affect scoring throughout all rounds.

The game ends automatically after the 9th round. Crucially, this simplification retains the two signature challenges of roguelike deck-builders that make them difficult for RL:

1) **Combinatorial explosion in Joker selection.** Even with 30 implemented types and 5 slots, the joint space

of subsets and orderings remains vast (Section II-B), preserving the strategic depth of the original game.

2) **Long-horizon, sparse rewards.** A single Joker choice propagates through all 9 subsequent rounds, yet the agent receives only a cumulative score at the end of the run. This delayed credit-assignment problem is characteristic of roguelikes, where early decisions compound over extended play horizons.

By stripping away shop economics and blind selection, we isolate these core difficulties and create a tractable testbed for investigating how LLM priors can accelerate RL in combinatorially rich, sparse-reward settings.

III. Environment Design

Rather than hooking into the original Lua-based game, we build a *custom simplified environment* in pure Python that reproduces core Balatro mechanics. The environment is not a faithful replica of the full game: it implements the simplified two-phase model from Section II (joker selection followed by 9 rounds of hand play), omitting shop economics, blind selection, consumables, and multi-ante progression. This design choice prioritizes fast, deterministic simulation for large-scale RL training over completeness.

A. Hand-Play Environment

The hand-play phase is formulated as an episodic MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

State Space. A 57-dimensional vector encoding 8 card slots (8×6: suit, rank, chip value, debuff, face-card, existence, all normalized) and game state (9 dims: hands/discards remaining, hand/deck size, score).

Action Space. Bitmask encoding (Discrete 510): actions 0–254 encode **play** with card mask $m = \text{action}+1 \in \{1, ..., 255\}$ (bit i selects card i), and actions 255–509 encode **discard** with the same mask.

Action Masking. A boolean mask enforces card existence, at-most-5 selection, and hand/discard availability.

Reward. Sparse terminal reward: $R_t = 0$ for non-terminal steps, $R_T = \text{total_score}$ (cumulative score over all 9 rounds), with $\gamma = 1$.

B. Joker Selection Environment

A separate Gymnasium environment handles joker selection. At the start of each episode, the environment samples 5 Jokers from the 30 implemented types. The agent observes their attributes (89-dimensional vector encoding joker effects and game context) and selects which to equip (up to 5 slots). The chosen jokers persist throughout all 9 rounds, affecting every scoring calculation.

IV. Method

Our framework assigns a specialized agent to each game phase: Section IV-A compares DDQN and PPO for hand-play, and Section IV-B describes the PPO agent with hesitation-gated LLM prior for joker selection.

A. Hand-Play Agents

We evaluate two RL approaches for hand-play, both using the bitmask action encoding and action masking from Section III-A.

DDQN. Double DQN [7] with action masking: invalid actions receive $Q = -\infty$ during both selection and ϵ-greedy exploration. Since $\gamma = 1$ with sparse terminal reward, we use Monte Carlo returns—every transition target is simply $y_t = R_T$—eliminating bootstrapping error.

PPO with Action Masking. PPO [8] with a masked categorical distribution: invalid logits are set to $-\infty$ before softmax, ensuring zero probability on illegal moves [9]. Both agents use identical network capacity to enable fair comparison of value-based versus policy-based methods.

B. Joker Agent: PPO with Hesitation-Gated LLM Prior

Joker selection requires strategic knowledge about effect synergies that is abundant in game guides but expensive to learn from scratch. We incorporate this knowledge by treating an LLM as a Bayesian action prior within a PPO framework [6], gated by a novel confidence-based mechanism.

1) LLM as Action Prior via RAG: We construct an action prior $p_{\text{LLM}}(a|s)$ using a language model augmented with Retrieval-Augmented Generation (RAG) [5]. Given state s (a text description of available jokers, deck composition, and current ante), the system:

1) Retrieves relevant passages from a curated knowledge base of Balatro strategy guides.
2) Prompts the LLM with the retrieved context and state, asking it to select the best action.
3) Repeats the query N times independently. The empirical action frequencies form the prior: $p_{\text{LLM}}(a|s) = n_a/N$, where n_a is the number of times action a was chosen across N queries.

2) Bayesian Framework: Variational Inference with LLM Prior: Following [6], [10], we frame RL as Bayesian inference. An optimality variable $\mathcal{O}$ indicates trajectory quality with likelihood $p(\mathcal{O}{=}1 \mid \tau) \propto \exp\big(\sum_t \gamma^t r_t/\alpha\big)$. The trajectory prior factorizes as:

$$p(\tau) = p(s_0) \prod_{t=0}^{T} p_{\text{LLM}}(a_t|s_t)\, P(s_{t+1}|s_t, a_t) \tag{2}$$

Using variational inference with $q(\tau) = p(s_0) \prod_t \pi_\theta(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t)$ as the variational distribution, maximizing the ELBO yields:

$$\max_\theta \; \mathbb{E}_{\pi_\theta}\Big[\sum_t \gamma^t r_t - \alpha \; \text{KL}\big[\pi_\theta(\cdot|s_t) \big\| p_{\text{LLM}}(\cdot|s_t)\big]\Big] \tag{3}$$

This objective balances reward maximization with proximity to the LLM prior, regularizing the policy toward expert-like behavior while permitting adaptation through online interaction.

3) Hesitation Gate: Invoking the LLM at every step is costly, and when the policy is already confident the prior may add noise rather than useful signal. We propose a *hesitation gate* that activates the LLM prior only when the policy exhibits uncertainty.

Motivation. The key observation is that a policy's softmax output reveals *how much it knows*. When the variance across action probabilities is low, the distribution is nearly uniform—the agent assigns similar probability to many actions and has no strong preference. This "hesitation" signals that the agent lacks sufficient knowledge to distinguish good actions from bad ones, which is precisely the situation where external guidance from the LLM is most valuable. Conversely, when the variance is high—indicating a peaked distribution with a clear preferred action—the agent has already learned a confident strategy, and injecting the LLM prior would only add noise or computational overhead. The hesitation gate formalizes this intuition: *consult the LLM when the policy hesitates, trust the policy when it is confident*.

Confidence measure. Given the PPO policy's softmax output $\pi_\theta(\cdot \mid s) = [p_1, \ldots, p_n]$ where $n = |\mathcal{A}|$, we compute the normalized variance:

$$h(s) = \frac{\mathrm{Var}\big(\pi_\theta(\cdot|s)\big)}{\sigma^2_{\max}}, \quad \sigma^2_{\max} = \frac{n-1}{n^2} \tag{4}$$

where $\mathrm{Var}(\pi) = \frac{1}{n}\sum_{i=1}^{n}(p_i - \frac{1}{n})^2$ and $\sigma^2_{\max}$ is the variance of a one-hot distribution. The measure $h(s) \in [0,1]$: $h = 0$ for a uniform distribution (maximum uncertainty) and $h = 1$ when all mass is on a single action (maximum confidence).

Gate activation. The gate function is:

$$g(s) = \begin{cases} 1 & h(s) < \tau \ \text{(uncertain — use LLM)} \\ 0 & h(s) \geq \tau \ \text{(confident — skip LLM)} \end{cases} \tag{5}$$

where $\tau \in (0,1)$ is a threshold hyperparameter.

Combined objective. Incorporating the hesitation gate into the PPO framework, the policy is optimized by:

$$\begin{aligned} L(\theta) = \mathbb{E}_{\pi_{\theta_{\text{old}}}}\Big[& \mathrm{clip}\big(r_t(\theta)\,\hat{A}_t,\ 1\pm\epsilon\big) \\ & + \beta\, H\big(\pi_\theta(\cdot|s_t)\big) \\ & - g(s_t)\,\alpha\ \mathrm{KL}\big[\pi_\theta(\cdot|s_t)\big\|p_{\text{LLM}}(\cdot|s_t)\big]\Big] \end{aligned} \tag{6}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio, $\hat{A}_t$ is the estimated advantage, β controls entropy regularization, and α weights the LLM prior influence. When the gate is off ($g{=}0$), this reduces to standard PPO; when on ($g{=}1$), the KL term pulls the policy toward the LLM's recommendations.

Intuition. A near-uniform softmax signals that the policy cannot distinguish between actions—precisely when external knowledge is most valuable. As training progresses and confidence grows, the gate activates less frequently, naturally transitioning from LLM-guided to autonomous behavior.

Algorithm 1 Hesitation-Gated PPO with LLM Prior

```
Initialize policy π_θ, value function V_φ
Load RAG knowledge base from game guides
for each iteration do
    Collect trajectories using π_θ
    for each state s_t in batch do
        Compute h(s_t) via Eq. 4
        if h(s_t) < τ then
            Query LLM with RAG: p_LLM(· | s_t)
            Compute KL[π_θ(· | s_t) || p_LLM(· | s_t)]
        else
            Set KL term = 0
        end if
    end for
    Update θ by maximizing L(θ) (Eq. 6)
    Update φ by minimizing value loss
end for
```

V. Experiments

Our experiments address three questions: (1) How do DDQN and PPO compare on combinatorial hand-play? (2) Does the hesitation-gated LLM prior improve joker selection over pure PPO? (3) Does the full two-phase pipeline outperform single-phase baselines?

A. Experimental Setup

All experiments use our pure-Python engine on consumer hardware. Both DDQN and PPO agents use identical network capacity, sparse terminal reward with $\gamma = 1$, and are trained for 1M episodes.

B. Experiment 1: Hand-Play Phase (DDQN vs. PPO)

DDQN and PPO with action masking are compared against a random baseline (uniform over valid actions). Both agents use sparse terminal reward with $\gamma = 1$ and identical network capacity. Metrics: mean terminal score, maximum score, and sample efficiency (episodes to reach 90% of final performance). A reward ablation compares sparse terminal reward against shaped per-step reward.

C. Experiment 2: Joker Selection with LLM Prior

The hesitation-gated LLM prior is evaluated on joker selection by comparing four conditions:

- **Random**: Uniform random joker selection.
- **PPO**: Standard PPO without LLM guidance.
- **PPO + LLM Prior**: PPO with constant KL regularization toward the LLM prior (Eq. 3, no gate).
- **PPO + Hesitation Gate**: Our full method (Eq. 6), with LLM prior activated only when policy confidence is low.

Performance is measured by total score over 9 rounds with a fixed trained hand-play agent. A central goal of this experiment is to quantify the *efficiency–performance trade-off* of the hesitation gate: we report (1) **LLM query reduction**—the percentage of LLM calls saved by the gated method compared to the always-on prior (PPO + LLM Prior), and (2) **performance retention**—the fraction of the always-on prior's score achieved by the gated variant despite fewer queries. We additionally track gate activation rate over training to verify the expected

transition from LLM-guided to autonomous behavior: early in training, when the policy is near-uniform and the gate fires frequently, and late in training, when the policy has converged and the gate activates rarely.

D. Experiment 3: Hesitation Gate Ablation

We study sensitivity to three hyperparameters:

- **Threshold** τ: We sweep $\tau \in \{0.1, 0.2, 0.3, 0.5, 0.7\}$ to characterize the trade-off between LLM query frequency and performance.
- **KL weight** α: We test $\alpha \in \{0.01, 0.1, 0.5, 1.0\}$ to measure the influence strength of the LLM prior.
- **Confidence measure**: We compare our normalized-variance measure against entropy-based and max-probability-based alternatives.

E. Experiment 4: End-to-End Evaluation

The full two-phase pipeline is evaluated end-to-end: a trained joker agent (with or without LLM prior) feeds its selections into a trained hand-play agent for 9 rounds. Comparison against single-phase baselines (random jokers, random play) quantifies the synergy between the two trained components.

VI. Related Work

RL for games. Deep RL has progressed from Atari [1] through strategy games like StarCraft II [2] and Dota 2 [11] to open-ended environments such as NetHack [12], Minecraft [13], and Pokémon Red [14]. Card games introduce stochasticity and imperfect information: Pluribus [3] achieved superhuman poker play, and Hanabi [4] tests cooperative reasoning under partial observability. Hearthstone has been approached with MCTS [15] and Slay the Spire with search methods [16]. Balatro differs from these as a *single-player* roguelike deckbuilder where the primary challenge is combinatorial optimization over card subsets and modifier selection—no opponent modeling is required, but strategic knowledge about card synergies is essential.

Incorporating external knowledge into RL. One line of work uses LLMs as reward shapers or value guides [17]; another treats LLM outputs as action priors within a Bayesian framework [6], [10]. We follow the latter approach but introduce two novelties: (1) a hesitation gate that conditions LLM engagement on policy confidence, reducing unnecessary queries, and (2) application to a card game domain where RAG over human-authored strategy guides provides the prior knowledge. Action masking [9] is orthogonal and critical for sample efficiency in our large discrete action space; we apply it in both DDQN and PPO agents.

VII. Discussion and Limitations

Hesitation gate design. Our normalized-variance gate is simple and interpretable but not the only option—entropy or max-probability measures are natural alternatives. The threshold τ trades off query cost against guidance benefit; adaptive thresholds that evolve during training are a promising direction.

LLM overhead. Each RAG query introduces latency. The hesitation gate reduces query frequency as training progresses, and policy distillation could further amortize LLM costs for real-time play.

Simplified game scope. Our environment implements 30 of 150+ Joker types and omits shop management, blind selection, and consumables. This suffices for validating the methodology, but a complete implementation is needed for human-level comparison.

Ongoing evaluation. The experimental framework is designed; full results are pending. We present planned experiments to enable reproducibility and invite community evaluation.

VIII. Conclusion

We presented an RL framework for Balatro that addresses different game phases with appropriate methods: DDQN and PPO are compared on the combinatorial hand-play phase, while a hesitation-gated LLM prior injects human game knowledge into joker selection exactly when the policy is uncertain. The hesitation gate provides a principled, confidence-aware mechanism for engaging LLM guidance that naturally reduces query frequency as training progresses.

Future work includes: (1) full experimental evaluation of the LLM-guided joker agent; (2) expanding to all 150+ Joker types with shop and blind selection phases; (3) end-to-end multi-phase training; and (4) comparison against human experts.

Our framework is publicly available to support research on RL for combinatorial card games.[1]

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[2] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev *et al.*, "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[3] N. Brown and T. Sandholm, "Superhuman AI for multiplayer poker," *Science*, vol. 365, no. 6456, pp. 885–890, 2019.

[4] N. Bard, J. N. Foerster, S. Chandar, N. Burch, M. Lanctot, H. F. Song, E. Parisotto, V. Dumoulin, S. Moitra, E. Hughes *et al.*, "The Hanabi challenge: A new frontier for AI research," *Artificial Intelligence*, vol. 280, p. 103216, 2020.

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[6] X. Yan, Y. Song, X. Feng, M. Yang, H. Zhang, H. Bou Ammar, and J. Wang, "Efficient reinforcement learning with large language model priors," *arXiv preprint arXiv:2410.07927*, 2024.

[7] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[9] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *The International FLAIRS Conference Proceedings*, vol. 35, 2022.

[1]Repository URL omitted for double-blind review.

[10] S. Levine, "Reinforcement learning and control as probabilistic inference: Tutorial and review," *arXiv preprint arXiv:1805.00909*, 2018.

[11] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Dębiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse *et al.*, "Dota 2 with large scale deep reinforcement learning," *arXiv preprint arXiv:1912.06680*, 2019.

[12] H. Küttler, N. Nardelli, A. Miller, R. Raileanu, M. Selvatici, E. Grefenstette, and T. Rocktäschel, "The NetHack learning environment," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 7671–7684.

[13] W. H. Guss, B. Houghton, N. Tobin, P. Wang, C. Codel, M. Veloso, and R. Salakhutdinov, "MineRL: A large-scale dataset of Minecraft demonstrations," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 2019, pp. 2442–2448.

[14] M. Pleines, D. Addis, D. Rubinstein, F. Zimmer, M. Preuss, and P. Whidden, "Pokémon Red via reinforcement learning," in *IEEE Conference on Games (CoG)*. IEEE, 2025.

[15] A. Santos, P. A. Santos, and F. S. Melo, "Monte Carlo tree search experiments in Hearthstone," in *IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 2017, pp. 272–279.

[16] E. Saavedra *et al.*, "Exploring the impact of randomness in roguelike deck-building games: A case study of Slay the Spire," in *International Symposium on Integrated Uncertainty in Knowledge Modelling and Decision Making*. Springer, 2023, pp. 95–107.

[17] S. Zhang, S. Zheng, S. Ke, Z. Liu, W. Jin, J. Yuan, Y. Yang, H. Yang, and Z. Wang, "How can LLM guide RL? a value-based approach," *arXiv preprint arXiv:2402.16181*, 2024.